Exp12 **write java proram to perform Ariori Algorithm**

Program:

```
import java.util.*;

public class SimpleApriori {

   // Method to calculate frequent itemsets
   public static void apriori(List<String[]> transactions, double minSupport) {
      // Step 1: Count item frequency
      Map<String, Integer> itemCount = new HashMap<>();
      for (String[] transaction : transactions) {
         for (String item : transaction) {
            itemCount.put(item, itemCount.getOrDefault(item, 0) + 1);
         }
      }

      // Step 2: Print itemsets that meet the minSupport threshold
      int totalTransactions = transactions.size();
      for (Map.Entry<String, Integer> entry : itemCount.entrySet()) {
         double support = (double) entry.getValue() / totalTransactions;
         if (support >= minSupport) {
            System.out.println("Item: " + entry.getKey() + ", Support: " + support);
         }
      }
   }

   // Main method to test the Apriori implementation
   public static void main(String[] args) {
      // Example dataset of transactions (each transaction is an array of items)
      List<String[]> transactions = new ArrayList<>();
      transactions.add(new String[]{"milk", "bread"});
      transactions.add(new String[]{"milk", "diaper", "beer", "bread"});
      transactions.add(new String[]{"milk", "diaper", "bread"});
      transactions.add(new String[]{"milk", "bread"});
      transactions.add(new String[]{"diaper", "beer", "bread"});

      double minSupport = 0.6; // Minimum support as a percentage (60%)

      // Run the Apriori algorithm
      apriori(transactions, minSupport);
   }
}
```

Output:

```
Item: Bread, Support:1.0
Item: Milk, Support: 0.8
Item: Diaper, Support : 0.6
```

Exp13: Write a R program to cluster your choice of data using simple k-means algorithm using JDK

Program:

```
# Step 1: Create a small dataset (4 points with 2 attributes)
data <- data.frame(
  x = c(1, 2, 3, 5),
  y = c(2, 3, 4, 6)
)

# Step 2: Set seed for reproducibility
set.seed(0)

# Step 3: Apply K-Means clustering with 2 clusters
kmeans_result <- kmeans(data, centers = 2)

# Step 4: Display cluster centers
print("Cluster Centers:")
print(kmeans_result$centers)

# Step 5: Display cluster assignment for each point
print("Cluster Assignment for each point:")
print(kmeans_result$cluster)
```

Output:

```
[1] "Cluster Centers: "
  x y
1 5 6
2 2 3
[1] "Cluster Assignment for each point: "
[1] 2 2 2 1
```

Exp 14: **Write a program of cluster analysis using simple k-means algorithm Python/R programming language.**

Program:

```
from sklearn.cluster import KMeans
import numpy as np
#-------

data = np.array([
    [1, 2], [1.5, 1.8], [5, 8],
    [8, 8], [1, 0.6], [9, 11]
])
#--------

kmeans = KMeans(n_clusters=2, random_state=0)
kmeans.fit(data)

#---------
print(kmeans.cluster_centers_)
print(kmeans.labels_)
```

Output:

[[7.33333333 9.]
[1.16666667 1.46666667]]

[1 1 0 0 1 0]

Exp15: **Write a program to compute/display dissimilarity matrix (for your own dataset containing at least four instances with two attributes) using Python**

Program:

```
import numpy as np

# Dataset: 4 instances with 2 attributes
data = np.array([
    [1, 2],
    [2, 3],
    [3, 4],
    [5, 6]
])
#------
# Compute dissimilarity (Euclidean distance)
n = len(data)
dissimilarity = np.zeros((n, n))

print(dissimilarity)
#--------


for i in range(n):
    for j in range(n):
        d = np.linalg.norm(data[i] - data[j])  # Euclidean distance
        dissimilarity[i, j] = round(d, 2)


#--------
# Display
print("Dissimilarity Matrix:")
print(dissimilarity)
```

Output:

[[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]]

Dissimilarity Matrix:
[[0. 1.41 2.83 5.66]
[1.41 0. 1.41 4.24]
[2.83 1.41 0. 2.83]
[5.66 4.24 2.83 0.]]